

教育对话主动服务

把高成本理解前移到后台，用下一轮最新 Query 做实时确认，再安全注入 Main

公司：阿里巴巴 | 角色：算法与工程设计【具体岗位待确认】 | 状态：steering 主链已具备，可靠交付/学习闭环待完善

30 秒版本 我设计的是教育对话里的主动引导，不是弹窗推荐。上一轮回答结束后异步用画像和会话跑 L0-L4，生成一个 pending steering；下一轮用户发来最新问题时，再用轻量 Delivery Gate 判断旧候选是否仍合适。只有配置开启且 Router 实际启动 Main 才注入。这样把两次主模型放到后台，实时路径只承担一次轻量 fit 判断，同时用规则、频控和 post-correction 保证主动内容不中断显式需求。

一、业务目标与边界

- 目标：在不打断主任务的前提下，基于画像、历史、掌握度和当前话题发现高价值延伸机会。
- 不是最终 Router：不能决定 Main/SubAgent owner，也不创建 Handoff Task。
- 不是最终回答：steering 是 Main 可忽略的内部建议，最新用户 Query 永远优先。
- 不是每轮必推：正常 miss、quiet turn 和异常时无主动内容都是正确结果。
- 不是已完成的在线学习系统：当前缺 durable delivery ledger、完整归因、蒸馏与回滚闭环。

二、我的工作

- 设计 prepare-confirm 两阶段架构，拆分后台 Graph 与下一轮 Delivery Gate，解决候选跨轮过时间题。
- 设计 L0-L4 分层：状态重建、规则准入、用户姿态感知、Goal+Guidance 联合决策、候选暂存/分发。
- 把模型输出转成可执行候选：危机/拒绝阻断、cooldown、通道可用性、知识点去重、结构与内容红线校验。
- 完成 EduBot 侧集成边界设计：Prompt 配置、Handoff Gate、Proactive Gate 并行；Main 注入四条件；失败不阻塞主回答。
- 梳理多 Worker / 多机房下的 Redis 水合、短锁与异步多写，定位重复消费、goal 丢失和 deadline 状态分叉。
- 结合 A/B 结果评估真实收益，并明确 Gate fit、Prompt 注入、语义消费和最终效果之间不可跳步。

三、端到端业务链路

- 第 n 轮结束
- > EduBot 异步 POST /events (门铃, 不携带完整真值)

-> 主动服务重读画像/Session -> L0 -> L1 -> L2 -> L3 -> L4

-> PendingSteering 写入进程内单槽 + Redis 镜像
- 第 n+1 轮 Query 到来
- > 同步 POST /gate (约 200ms 调用侧预算)

-> 水合 + detach -> 轻量 0/1 fit judge -> fit 返回 steering/behavior

-> Router 决定本轮 target

-> 仅 Prompt 开关 on + steering 非空 + 实际执行 Main 时注入

-> Main 独立回答, 可融合/忽略/ToolCall Handoff

核心思想 候选生成时的上下文属于上一轮；下一轮用户可能换题、拒绝或进入 SubAgent。主动服务必须让最新 Query 否决旧候选，因此不能在生成后立即展示。

四、L0-L4 算法设计

层	模型成本	职责	失败/阻断语义
L0 ContextBuild er	0	重读画像与历史, 重建描述状态、turn、handoff	部分数据可降级; 不从空 event 编 造
L1 RuleGate	0	kill switch、cold start、cap、双通道 cooldown	全部通道不可用则 quiet, 零模型 成本
L2 Observer	1 次主模型	描述 topic、act、engagement、opportunity、 情绪/危机	解析失败静默结束; 危机/拒绝确 定性阻断
L3 Reasoner	1 次主模型	联合决策 engage、goal action、guidance、 channel	post-correction 删除非法 intent; 无 surviving intent 不推 进 goal
L4 Dispatcher	0	投影 Main view, 生成 pending, 写单槽/Redis	quiet 不写空候选, 不 tick 频控
Delivery Gate	1 次轻量判 断	用最新 Query 判旧候选 fit/unfit	error/timeout 对主动内容保守关 闭

4.1 为什么 L2 只描述、不决策

把 Perception 和决策分离可以复用感知结果，并让危机、拒绝、stop/shift 等红线由确定性代码复查。L2 只说明用户状态和机会强弱；L3 才结合 goal、频控、历史干预和可用通道决定是否介入。

4.2 为什么 L3 联合决策 Goal 与 Guidance

如果分别让多个模型定目标、写引导、选通道，容易产生方向冲突和额外成本。一次联合决策能保持 should_engage、goal_action、guidance 与 card_intent 一致，再由代码按状态机和安全规则校正。

4.3 post-correction

- 合法化 create/maintain/advance/revise/branch/drop；不存在 goal 时不能凭空 advance。
- should_engage=false 或 drop 时删除 guidance；cooldown 通道 intent 强制删除。
- 删除未开启 card phase 的幻觉输出；知识点冲突、方向不一致、必填字段缺失时降级 quiet。
- 拦截危机、健康焦虑、越界工具主题；没有 surviving intent 时不允许推进 goal。

五、工程设计

5.1 关键路径与 fail-open

- 两次主模型在上一轮后台运行，下一轮只做轻量 fit 判断；Event HTTP 不延长上一轮回答。
- 客户端按 Worker 复用连接，入口并发准备 PromptGet、Handoff Gate 与 Proactive Gate。
- 对主回答 fail-open：主动服务超时/Redis/模型异常时仍正常回答；对主动内容 fail-closed：未经确认不展示。
- Main 消费合同要求核心回答独立完整、最多自然融合一个短延伸、不得暴露内部策略。

5.2 多 Worker / 多机房缓存

- 进程内 SessionContext 是同 Worker 快路径；Redis 镜像解决 Event 在 A、Gate 在 B 的水合。
- Event 后台向多个机房并发写，Gate 默认只读本地，实时路径避免跨区 RTT；Redis 故障退化为 miss。
- Gate 只在短锁内 detach 槽位，模型 judge 放在锁外，避免实时请求等待后台模型。
- 当前 GET-judge-DELETE 不是全局原子，两个 Worker 可重复判断同一候选；多机房异步删除也有短暂重复窗口。

5.3 提交点和 deadline 错配

当前服务在 Gate fit 时就提交 candidate goal、tick clock 并删除候选，但这早于 EduBot 收到、Prompt 开关、Main 注入和用户可见。更危险的是调用侧约 200ms、服务 Judge 约 400ms：调用方可能已超时继续主回答，服务端却稍后把候选当作已消费。目标是 STAGED->CLAIMED->JUDGED->DELIVERED_ACKED/RELEASED 的原子状态机，并传播 deadline 或感知取消。

六、实验收益与正确解读

指标	On	Off	提升	p-value
Session 数	16,238	15,889	+349	-
实际注入 Session	417 (2.57%)	0	+417	-
平均 Main 轮次	3.310	2.830	+0.480	0.000008
多轮率 (>=2)	27.26%	25.36%	+1.91pp	0.000105
深度对话率 (>=5)	11.66%	10.01%	+1.66pp	0.000002
平均会话时长	176.54s	163.87s	+12.67s	0.0096

怎么解释 这是 on/off 的意向治疗口径，只有 2.57% Session 实际注入。不能把整体 lift 直接解释成“每次注入的因果效果”，还需检查随机化、曝光归因、守护指标和异质性。面试时先给结果，再主动说这一限制，可信度会更高。

七、现状、技术债与演进

当前已有	关键缺口	目标方案
Event + full fetch	缺 turn/version/high-watermark 因果锚点	durable inbox；数据版本写入 decision snapshot
内存 + Redis pending	跨 Worker 非原子 claim；多机房重复窗口	Lua/CAS atomic claim + delivery token + lease
Gate fit 延迟提交	仍早于真实注入；goal 字段水合不完整	candidate goal 完整持久化；Injection/User-visible ACK
Feedback API / Reflector 局部能力	缺 durable ledger、归因与策略回滚	intervention ledger -> 离线评分/审计 -> 版本化蒸馏 -> A/B/回滚

当前已有	关键缺口	目标方案
Card 状态机框架	主端 receiver、展示/点击反馈未闭环	幂等 receiver + 展示 ACK + attribution

八、面试高频问答

Q1. 为什么要两阶段，而不是上一轮直接推？

建议回答：候选基于上一轮上下文，下一轮用户可能换题、拒绝或进入 SubAgent。后台 prepare 降低实时成本，实时 confirm 让最新 Query 拥有否决权。

Q2. 为什么 Event 只是门铃？

建议回答：完整画像和会话保存在权威数据源，Event 只携带身份和触发信息可降低耦合、重复时也能收敛到同一视图；代价是要处理数据落盘可见性和版本锚点。

Q3. L1 为什么不做情绪关键词规则？

建议回答：危机、拒绝和情绪需要语境，简单关键词误判高。L1 只做确定性的开关/频控准入，L2 模型感知后再用代码红线复查。

Q4. 为什么 L2 失败要 quiet？

建议回答：主动介入属于附加能力，错误干预的风险高于没有干预。解析失败时伪造默认 Perception 会为了产出率牺牲安全。

Q5. 为什么 Goal 不能在 L4 生成时就提交？

建议回答：候选还可能过期或被下一轮否决。若提前推进 goal，系统会误以为已经教过。至少要延迟到交付定义成立，理想是注入或用户可见 ACK。

Q6. Gate fit 为什么还不等于成功？

建议回答：后面还有网络返回、Prompt 开关、Router 是否执行 Main、Prompt 是否注入、Main 是否采用、最终 responder 和用户效果。每层只能证明自己的阶段。

Q7. 为什么 steering 0 就丢弃，而 card 可以保留？

建议回答：steering 生成相对便宜且强依赖最近语境，旧建议反复尝试更危险；card 是较昂贵成品，fit=0 可能只是时机不对，TTL 内可继续等待。

Q8. Redis 为什么需要，为什么又不够？

建议回答：Redis 让跨 Worker 看见候选，但普通 GET/DELETE 不提供全局 claim；两个 Worker 仍可同时消费，多机房副本还会短暂重复，所以要有原子状态机和 delivery token。

Q9. 如何设计 atomic claim？

建议回答：用版本比较或 Lua 把 STAGED 原子转 CLAIMED，写 request_id、delivery_token 和 lease deadline；judge 后转 FIT/UNFIT，收到下游 ACK 才 DELIVERED，否则释放或 lease 超时回收。

Q10. 调用侧 200ms、服务侧 400ms 有什么问题？

建议回答：调用方先超时后服务端仍可能 detach/提交候选，造成用户没收到但服务认为已消费。要传播 deadline、支持取消，或让 caller budget 覆盖网络/排队/judge/响应余量。

Q11. 怎样证明 Main 真用了 steering？

建议回答：把 intervention_id 贯穿 Gate 响应、Prompt 组装、Main 执行与 final responder；对 Main 文本/ToolCall 做语义消费判定，并和最新 Query、最终可见内容、后续行为关联。

Q12. A/B 结果怎么讲才严谨？

建议回答：先给样本量、实际注入率和四个显著 lift，再说明它是 on/off 总体口径，不等于 treated-only 因果效应；需要随机化检查、守护指标、曝光日志和分人群分析。

Q13. 频控为什么按通道分开？

建议回答：steering 和 card 的成本、打扰程度与交付机制不同，应该各有 count 和 last_output_turn；候选生成/miss 不消耗额度，真实交付才 tick。

Q14. 怎么保证最新用户意图优先？

建议回答：Delivery Gate 用最新 Query 重判，Main 的 behavior contract 要求先完整回答显式需求；危机、拒绝、shift 信号阻断，且 SubAgent 路由不注入 steering。

Q15. 如果让你继续做，PO 是什么？

建议回答：先修交付状态一致性：完整持久化 candidate goal，原子 claim + lease + ACK，统一 deadline；然后补 intervention ledger，才能可靠频控、归因和策略学习。

九、两分钟项目陈述模板

这个项目做的是教育对话中的主动引导，但我们不把它当即时推荐。因为上一轮结束时生成的建议，到下一轮可能已经过时，所以我把链路设计成 prepare-confirm：上一轮结束后 EduBot 异步发 Event，主动服务重读画像和 Session，经过 L0 状态重建、L1 规则准入、L2 用户姿态感知、L3 Goal 与 Guidance 联合决策、L4 暂存，得到 pending steering；下一轮用户发来最新 Query 时，再由轻量 Delivery Gate 做 0/1 fit 判断。只有请求配置开启、Gate 返回非空且 Router 实际启动 Main 时才注入，Main 仍要先完整回答用户，必要时可以忽略或转交 SubAgent。工程上我们把两次主模型放到后台，实时路径只做一次轻量判断，并用连接复用、Redis 跨 Worker 水合、TTL 和短锁控制时延。但我也识别到当前 GET-judge-DELETE 不是全局原子、candidate goal 跨 Worker 不完整，以及调用侧 200ms 小于服务侧 400ms 的状态分叉。对应目标是 atomic claim、delivery token、lease 和下游 ACK。实验中 On/Off 为 16,238/15,889 Sessions，实际注入率 2.57%，平均 Main 轮次、多轮率、深度对话率和时长都有统计显著提升；我会把它解释为总体 on/off 结果，而不是把所有 lift 都直接归因到被注入样本。

资料依据与表达边界

边界 可以说 steering 生成与实时 Gate 主链已具备；完整可靠交付、Card 接入、durable feedback/reflector 和在线学习闭环仍是缺口。实验图展示总体 on/off 指标，treated-only 因果归因尚不足。

主要依据：

阿里实习材料/03-主动服务完整业务与算法设计.md；阿里实习材料/04-统一架构与链路工程设计.md；阿里实习材料/各类实验收益.txt；阿里实习材料/ec1cfe27bcc0e4264ce0bf7c53ee1d6d.jpg